



AI面接インタビューシステム

Day 5・Day 6 報告書

作成日: 2026年3月8日

リポジトリ: tianzhongyoushi178/AI_Interviewer

ブランチ: master

フレームワーク: Rails 6.1.7 / Ruby 3.1.6

目次

Day 5 — LLM制御レイヤーの実装

- 1. 概要とアーキテクチャ
- 2. プロンプトテンプレート管理
- 3. レスポンスバリデーション
- 4. LLMClient 強化（リトライ機構）
- 5. ResponseEvaluator 改修
- 6. コードレビュー結果
- 7. 変更ファイル一覧

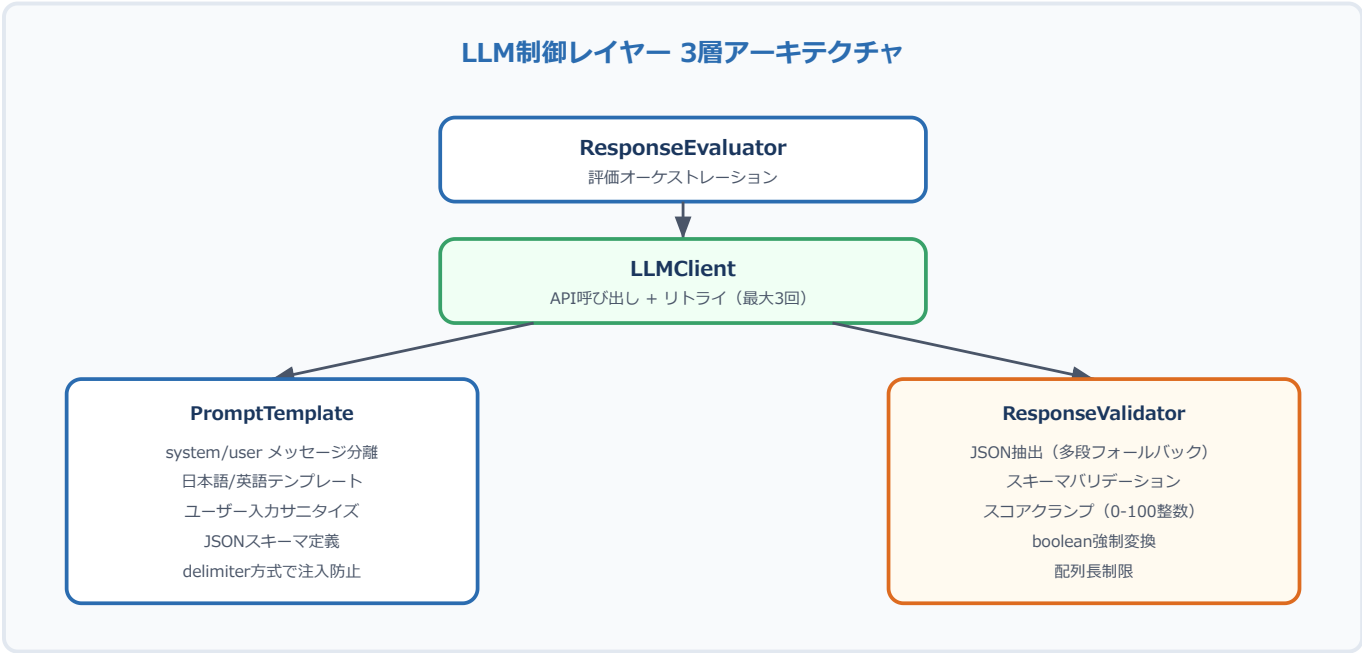
Day 6 — 面接セッション管理

- 8. 概要
- 9. DB変更（マイグレーション）
- 10. URL即時開始（トークンベース認証）
- 11. セッションタイムアウト
- 12. 中断復帰
- 13. 状態遷移バリデーション
- 14. セキュリティ対策
- 15. コードレビュー結果
- 16. 変更ファイル一覧
- 付録: 全APIエンドポイント一覧（Day 6時点）

Day 5 — LLM制御レイヤーの実装

1. 概要とアーキテクチャ

LLMの出力を厳密に制御する3層構造を構築した。従来は LLMClient にプロンプト生成・API呼び出し・レスポンス解析が混在していたが、責務を分離し、プロンプトテンプレート管理・レスポンスバリデーション・リトライ機構を独立したクラスとして実装した。



2. プロンプトテンプレート管理

ファイル: app/services/interview_engine/prompt_template.rb

設計方針

項目	方針
メッセージ構造	system (ロール制限) + user (タスク指示) の分離
言語対応	en / ja を normalize_language で安全に切替
自由生成排除	system promptに「JSON出力のみ・会話禁止・追加質問禁止」を明示
プロンプトインジェクション対策	delimiter方式 (---BEGIN USER INPUT--- / ---END USER INPUT---)

テンプレート一覧

テンプレート	メソッド	出力スキーマ
回答評価（記述式）	evaluation(question_type: 'open')	{relevance_score, correctness_score, clarity_score, final_score, passed, reasoning}
回答評価（選択式）	evaluation(question_type: 'choice')	同上
面接サマリー	summary(responses_data:)	{summary, strengths[], weaknesses[], recommendation}

サニタイズ処理

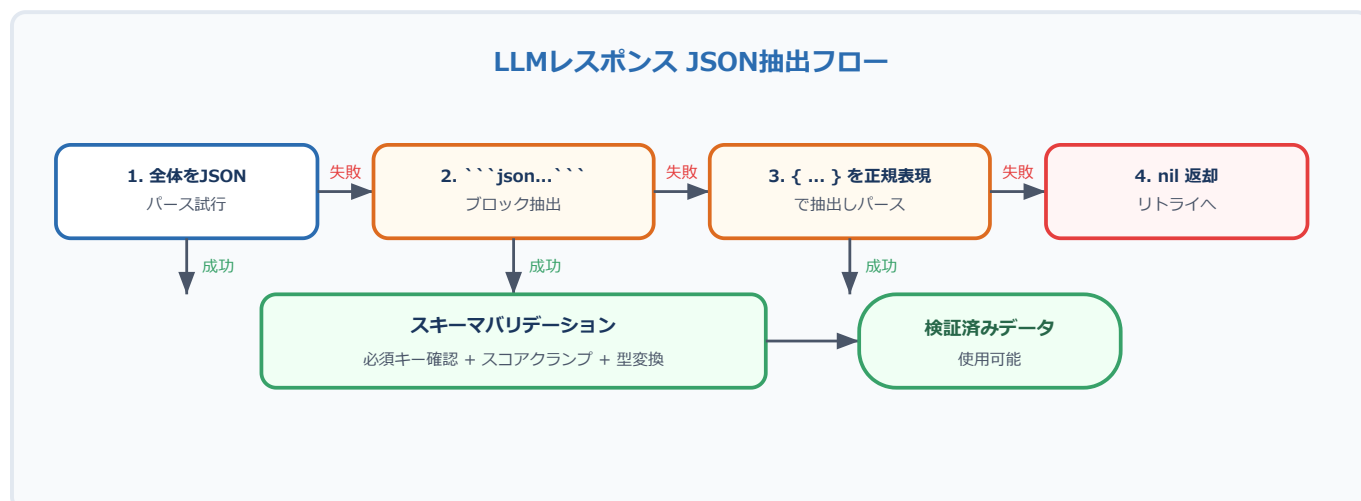
```
# ユーザー入力のサニタイズ（プロンプトインジェクション防止）
def sanitize(text)
  sanitized = text.to_s
  .gsub(/`"/, "'") # コードブロック記法のエスケープ
  .strip
  .truncate(2000) # 長文制限
  "---BEGIN USER INPUT---\n#{sanitized}\n---END USER INPUT---"
end
```

delimiter方式を採用した理由: 当初は {} のエスケープ方式を検討したが、技術系の回答でJSON記法やハッシュ構文を含む正当な入力を破壊するリスクがあったため、入力領域を明示的に区切るdelimiter方式に変更した。

3. レスポンスバリデーション

ファイル: app/services/interview_engine/response_validator.rb

JSON抽出フロー（多段フォールバック）



バリデーション処理一覧

処理	対象	内容
スキーマ検証	評価/サマリー共通	必須キーの存在チェック (data.key? で JSON null も許容)
スコアクランプ	評価のみ	0-100範囲に制限、整数に丸め (.to_f.round)
型変換	評価のみ	passed を boolean に強制変換
配列制限	サマリーのみ	strengths/weaknesses を最大5項目・各200文字に制限

レビュー修正: 当初 data[key].nil? で必須キーチェックを行っていたが、LLMがJSON null を返した場合に「キーが存在しない」と誤判定される問題があった。data.key?(key) に変更し、null値を持つキーも正しく認識するようにした。

4. LLMClient 強化 (リトライ機構)

ファイル: app/services/interview_engine/llm_client.rb

リトライ設定

設定	値
最大リトライ回数	3回
バックオフ	指数 (1秒ベース)
リクエストタイムアウト	30秒 (接続/読取り)

エラー分類とリトライ判定



モデル設定（環境変数）

環境変数	デフォルト	説明
LLM_MODEL	openai	使用モデルプロバイダ（openai/claude）
OPENAI_MODEL	gpt-4	OpenAI モデル名
CLAUDE_MODEL	claude-sonnet-4-20250514	Claude モデル名

OpenAI固有の制御

- `response_format: { type: 'json_object' }` -- JSON出力を強制
- `temperature: 0.2` -- 出力の安定性を重視
- `http.use_ssl = uri.scheme == 'https'` -- 動的SSL判定（テスト環境対応）

5. ResponseEvaluator 改修

ファイル: `app/services/interview_engine/response_evaluator.rb`

変更	詳細
LLM呼び出し分離	<code>llm_evaluate</code> メソッドに抽出
トランザクション制御	評価結果保存 + 面接継続判定を <code>ActiveRecord::Base.transaction</code> で保護
スコア再計算の明示	LLMの <code>final_score</code> を無視し、加重平均で再計算する理由をコメントで明記
型安全性	<code>@response.final_score.to_f</code> でJSON store経由の文字列型に対応
戻り値統一	全評価パスで <code>with_indifferent_access</code> を使用

スコア計算式

```
# 加重平均によるfinal_score計算
# LLMが返すfinal_scoreは信頼性が不安定なため、サーバー側で再計算
final_score = (relevance * 0.4) + (correctness * 0.4) + (clarity * 0.2)
passed = final_score >= 70
```

6. コードレビュー結果

重要度	件数	主な内容
Critical	5	JSON store型問題、リトライ対象漏れ、キーチェック方式、sanitize方式、メインスレッド sleep
Warning	10	N+1クエリ、スコア丸め、トランザクション分断、戻り値型不統一
Info	4	require配置、DI検討、設計メモ

Critical修正の詳細

#	問題	修正内容
1	JSON storeの final_score が文字列で返される可能性	.to_f で明示的に数値変換
2	429/5xxエラーがリトライ対象外だった	LLMResponseError を追加し call_with_retry で捕捉
3	data[key].nil? がJSON nullを未存在と誤判定	data.key?(key) に変更
4	sanitize が {} 置換で技術回答を破壊	delimiter方式に変更
5	メインスレッドでの sleep 懸念	RETRY_DELAY_BASE を2秒→1秒に短縮（Sidekiqジョブ内で実行前提）

7. 変更ファイル一覧

app/services/interview_engine/prompt_template.rb

新規 プロンプトテンプレート管理（EN/JA対応、スキーマ定義、サニタイズ）

app/services/interview_engine/response_validator.rb

新規 LLM出力バリデーション（JSON抽出、スキーマ検証、クランプ）

app/services/interview_engine/llm_client.rb

改修 リトライ機構、タイムアウト、エラー分類、モデル設定柔軟化

app/services/interview_engine/response_evaluator.rb

改修 トランザクション、型安全性、メソッド分離

app/services/interview_engine/session_manager.rb

改修 N+1修正（includes）、キーアクセス統一

Day 6 — 面接セッション管理

8. 概要

3つの機能を実装し、面接セッションのライフサイクルを完全に管理できるようにした。

機能	説明
URL即時開始	トークンベース認証により、Deviseログイン不要で面接を開始可能
セッションタイムアウト	一定時間操作がない面接を自動的にabandon状態に遷移
中断復帰	abandoned状態の面接を再開し、中断した箇所から続行

9. DB変更（マイグレーション）

ファイル: db/migrate/20260308120000_add_session_management_to_interviews.rb

interviews テーブル

カラム	型	デフォルト	説明
access_token	string	null	URL即時開始用トークン（UNIQUE INDEX）
last_activity_at	datetime	null	最終操作時刻
resumed_at	datetime	null	最終復帰時刻
resume_count	integer	0	復帰回数

situations テーブル

カラム	型	デフォルト	説明
session_timeout_minutes	integer	60	セッションタイムアウト（分）
allow_resume	boolean	true	中断復帰を許可するか
max_resume_count	integer	3	最大復帰回数

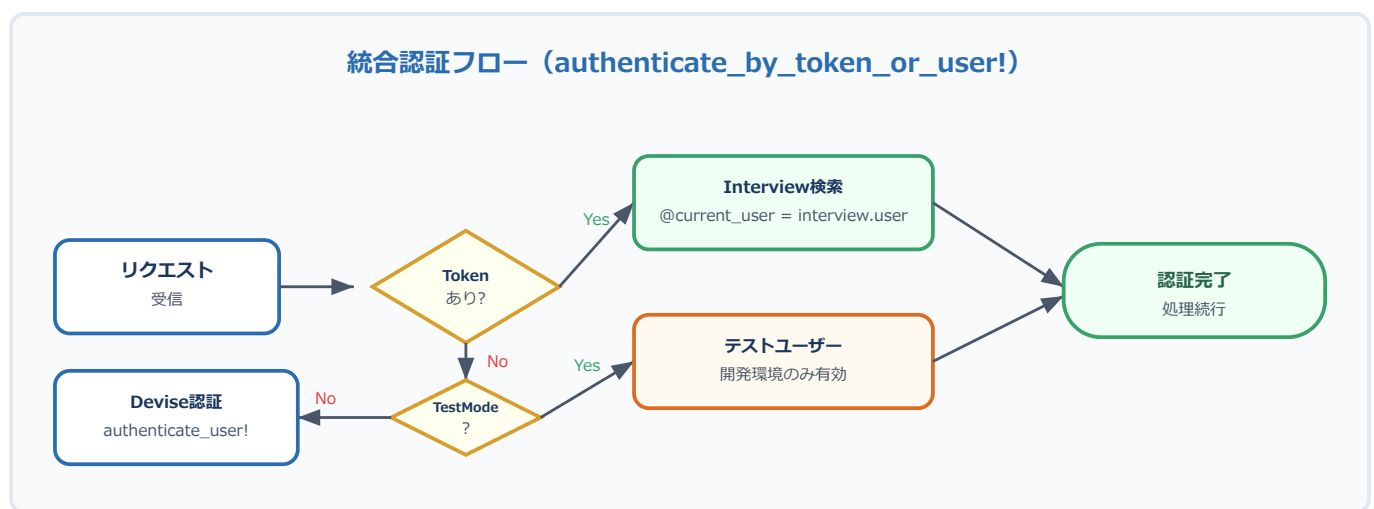
後方互換性: 既存データはデフォルト値が適用され、既存の面接フローに影響なし。マイグレーション適用のみで互換性が維持される。

10. URL即時開始（トークンベース認証）

トークン生成

```
# Interview 作成時に自動生成 (before_create コールバック)
def generate_access_token
  self.access_token = loop do
    token = SecureRandom.urlsafe_base64(32) # 43文字のURL-safeトークン
    break token unless Interview.exists?(access_token: token)
  end
end
```

認証フロー



新エンドポイント

メソッド	パス	認証	説明
POST	/api/interviews/start_by_token	トークンのみ	URL即時開始/自動復帰

レスポンス例

```
{
  "success": true,
  "interview_id": 42,
  "status": "in_progress",
  "total_questions": 10,
  "language": "ja",
  "progress": 30.0,
  "answered_questions": 3,
  "session_timeout_minutes": 60,
  "remaining_seconds": 3420,
  "resume_count": 0
}
```

運用想定

1. Client が Situation を作成
2. POST /api/interviews/start でInterviewを作成（access_token が返却される）
3. 受験者に https://example.com/interview?token=XXXXX のURLを送付
4. フロントエンドが start_by_token を呼び出し、認証不要で面接開始

11. セッションタイムアウト

設計

項目	内容
タイムアウト基準	last_activity_at からの経過時間（アイドルタイムアウト方式）
設定単位	Situation単位（Client が設定可能、1-180分）
チェックタイミング	next_question / submit_answer の before_action
タイムアウト時の遷移	in_progress → abandoned（410 Gone レスポンス）

タイムアウト判定

```
def timed_out?  
  return false unless in_progress? && last_activity_at.present?  
  timeout = situation.session_timeout_minutes.minutes  
  last_activity_at < timeout.ago  
end
```

アクティビティ更新タイミング

- start! -- 面接開始時
- next_question -- 質問取得時
- submit_answer -- 回答送信時
- resume! -- 面接再開時
- start_by_token -- トークン認証開始時

タイムアウト時のAPIレスポンス

```
{  
  "success": false,  
  "error": "Interview session has timed out",  
  "reason": "timeout",  
  "resumable": true  
}
```

バッチ処理（一括期限切れ）

```
# Sidekiq cronやRakeタスクから呼び出し
InterviewEngine::SessionManager.expire_timed_out_sessions!
```

SQLite互換性: 当初SQLのWHERE句でタイムアウト判定を行うスコープを実装していたが、SQLite固有の日時関数に依存するためレビューで指摘を受け、Ruby側の `timed_out?` メソッドのみで判定する方式に変更した。

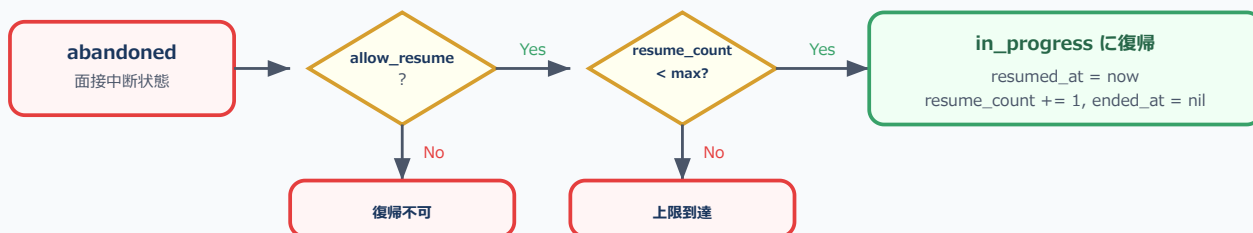
12. 中断復帰

復帰可能条件

```
def resumable?
  return false unless abandoned? || (in_progress? && timed_out?)
  return false unless situation.allow_resume?
  return false if resume_count >= situation.max_resume_count
  true
end
```

復帰フロー

面接中断復帰フロー



新エンドポイント

メソッド	パス	認証	説明
POST	/api/interviews/:id/resume	トークン or Devise	中断面接の再開

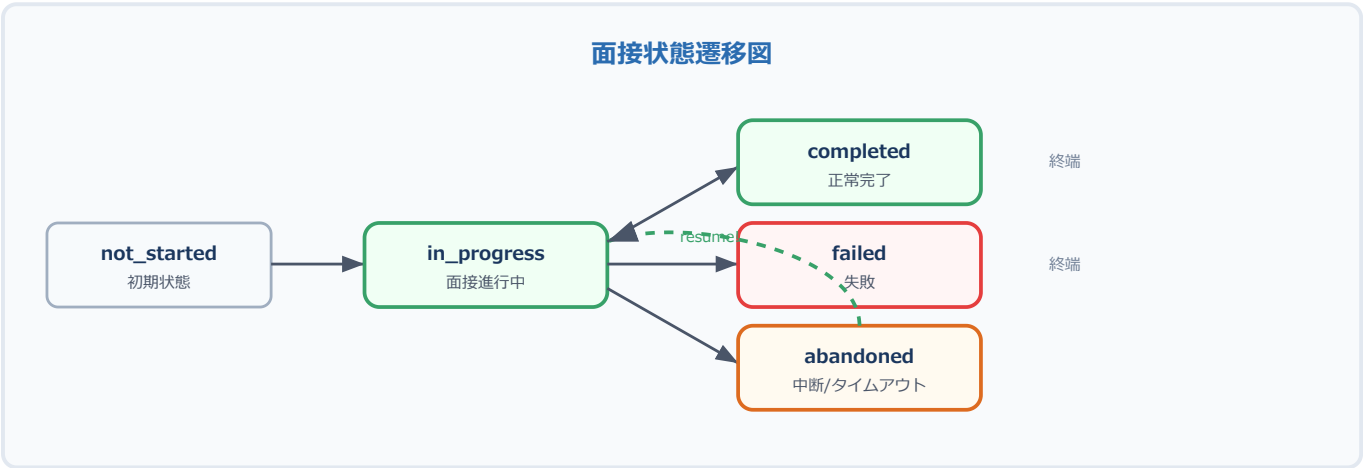
自動復帰パス

以下のケースでは明示的な resume 呼び出し不要:

呼び出し元	条件	動作
start_interview	abandoned面接が存在 & resumable	自動resume
start_interview	in_progress面接が存在	touch_activity して返却
start_by_token	abandonedのトークン	自動resume

13. 状態遷移バリデーション

不正な状態遷移をモデルレベルで防止する。



実装

```
VALID_TRANSITIONS = {
  not_started: [:in_progress],
  in_progress: [:completed, :failed, :abandoned],
  abandoned:  [:in_progress],
  completed:  [],
  failed:     []
}.freeze

validate :valid_status_transition, if: :status_changed?
```

14. セキュリティ対策

対策	内容
トークン強度	SecureRandom.urlsafe_base64(32) -- 256bit entropy
トークン一意性	DB UNIQUE INDEX + ループ生成で衝突回避
認可統合	トークン認証 → Devise認証のフォールバック
テストモード保護	Rails.env.production? でテストモードの本番有効化を防止
認可チェック強化	@current_user + user_id 比較 + nil ガード
レースコンディション	DB UNIQUE制約 + RecordNotUnique 捕捉 (409 Conflict)
状態遷移保護	VALID_TRANSITIONS バリデーションで不正遷移を防止
レート制限	Day 15セキュリティ強化フェーズで Rack::Attack を追加予定

15. コードレビュー結果

重要度	件数	主な内容
Critical	4	SQLite固有スコープ、認可バイパス、二重DB検索、レート制限欠如
Warning	8	レースコンディション、テストモード漏洩、状態遷移バリデーション、ロジック重複
Info	4	マイグレーションバージョン、タイムアウト基準確認、CSRF保護範囲

Critical修正の詳細

#	問題	修正内容
1	SQLite固有の日時関数を使用した timed_out スコープ	スコープ削除、Ruby側の timed_out? メソッドのみで判定
2	authorize_interview! が current_user (nil) を使用	@current_user + nil ガードに変更
3	validate_interview_exists と set_interview の二重DB検索	単一の set_interview に統合 (find_by + nil チェック)
4	start_by_token にレート制限なし	Day 15で Rack::Attack による制限を追加予定 (TODO コメント追加)

16. 変更ファイル一覧

db/migrate/20260308120000_add_session_management_to_interviews.rb

新規 access_token, last_activity_at, resumed_at, resume_count + Situation設定

app/models/interview.rb

改修 トークン生成、タイムアウト判定、中断復帰、状態遷移バリデーション

app/models/situation.rb

改修 セッション設定バリデーション

app/services/interview_engine/session_manager.rb

改修 トークン認証開始、復帰ロジック、タイムアウト一括処理

app/controllers/api/interviews_controller.rb

改修 統合認証、新エンドポイント、タイムアウトチェック、セキュリティ修正

config/routes.rb

改修 start_by_token, resume ルート追加

付録: 全APIエンドポイント一覧 (Day 6時点)

メソッド	パス	認証	概要
POST	/api/interviews/start	Devise/Token	面接開始 (既存面接の自動復帰対応)
POST	/api/interviews/start_by_token	Token のみ	URL即時開始/自動復帰
GET	/api/interviews/:id/next_question	Devise/Token	次の質問取得 (タイムアウトチェック付き)
POST	/api/interviews/:id/submit_answer	Devise/Token	回答送信 (タイムアウトチェック付き)
POST	/api/interviews/:id/complete	Devise/Token	面接完了
GET	/api/interviews/:id/status	Devise/Token	状態取得 (remaining_seconds, resumable含む)
POST	/api/interviews/:id/resume	Devise/Token	中断面接の再開